

WEEK 4 SUMMARY

Authentication, Identity, Input Validation, and API Security in ASP.NET Core

Program: Backend Development Internship (.NET)

Week: Week 4 of 10 | Phase 2: Core Technical Training

Theme: Authentication, Identity, Input Validation, and API Hardening

Duration: 40 hours across five training days

Executive Summary

Week 4 focused on transforming the REST API built in Week 3 into a production-oriented service protected by identity management, authentication, authorization, validation, and baseline security controls. The central outcome was an API that can securely register and authenticate users, issue and validate JWTs, enforce permissions, reject invalid requests early, and reduce exposure to common web threats.

The week emphasized that security controls are connected: password hashing protects stored credentials; JWT validation confirms each caller; authorization decides what that caller may do; input validation protects application logic and data quality; and rate limiting, CORS, HTTPS/HSTS, security headers, and parameterized database access provide defense in depth.

Learning Objectives Achieved

- Integrated ASP.NET Core Identity with Entity Framework Core for user management, roles, and secure password storage.
- Implemented JWT-based login and configured bearer authentication middleware.
- Protected routes with [Authorize], role-based access control, and a named authorization policy.
- Applied FluentValidation to create and update requests with consistent, meaningful validation errors.
- Hardened the API using rate limiting, a restrictive named CORS policy, HTTPS/HSTS, and security headers.
- Explained why EF Core LINQ queries are protected from SQL injection by default when parameterization is preserved.

Five-Day Curriculum and Practical Work

Day	Focus	Key Outcome
1	ASP.NET Core Identity	Registration, Identity schema, and secure password hashing.

Day	Focus	Key Outcome
2	JWT Authentication	Login flow, signed token issuance, and bearer-token validation.
3	Authorization & RBAC	Protected routes, roles, policies, and Postman token testing.
4	FluentValidation	Business-rule validators and structured 400 error responses.
5	API Hardening	Rate limits, CORS, HTTPS/HSTS, headers, and SQL-injection safety.

Day 1: Identity and Secure Registration

ASP.NET Core Identity was introduced as the preferred membership system because it provides user storage, password hashing, role management, and account-related capabilities on top of EF Core. The existing application DbContext is extended from IdentityDbContext, and a migration creates the required Identity tables alongside application entities.

The registration endpoint uses UserManager.CreateAsync so Identity performs password hashing and persistence safely. Passwords are hashed with PBKDF2 by default, using a unique salt per user. This makes hashes resistant to brute-force and rainbow-table attacks and reinforces the rule that custom password-hashing code should not be written when Identity already provides a tested implementation.

Hands-on result: Added Identity packages and services, updated the DbContext, applied the migration, built a registration endpoint, and tested successful registration and weak-password failures in Postman.

Day 2: JWT Authentication and Token Issuance

JWTs were used to authenticate requests without resending credentials. A JWT contains a header, a payload of claims, and a signature. Claims can identify the user and their roles, but the payload is readable by clients, so sensitive information must never be included.

The login endpoint verifies credentials with SignInManager.CheckPasswordSignInAsync. After a successful login, it creates a signed JWT containing safe claims such as the user ID and email. JWT bearer middleware then validates issuer, audience, signature, and expiry before a protected endpoint is reached. Invalid credentials return 401 Unauthorized.

- Use short-lived access tokens (for example, 15 minutes to a few hours) to limit the usefulness of stolen tokens.
- Treat the signing key like a database password: keep it out of source control and use local ignored settings or production secret management.
- Refresh tokens were introduced as an optional extension for renewing access without repeated credential entry.

Day 3: Protected Routes and Authorization

The [Authorize] attribute was applied at controller or action level to require a valid bearer token. Requests without a token, with an expired token, or with an invalid signature are rejected with 401

Unauthorized before application code runs. [AllowAnonymous] can be used selectively for public actions such as a health check.

Role-based access control differentiated at least two roles, User and Admin.

UserManager.AddToRoleAsync assigns roles, while [Authorize(Roles = "Admin")] restricts privileged operations such as deletion. A signed-in user without permission receives 403 Forbidden, which is distinct from 401 because authentication succeeded but authorization failed.

The week also introduced claims-based and policy-based authorization. Named policies centralize more complex rules and avoid scattering permission strings across endpoints, making future changes safer and more maintainable.

Day 4: Input Validation with FluentValidation

FluentValidation was selected for rules that go beyond simple model annotations. While DataAnnotations work for basic required fields and maximum lengths, FluentValidation keeps complex business rules in separate validator classes and supports cross-field logic, custom messages, and maintainable validation chains.

Validators were created for both Create and Update request models. Examples include requiring a positive CustomerId, ensuring an order contains at least one item, constraining a discount to 0-100, and requiring an end date after a start date. Once registered, validators execute during model binding so invalid requests return a structured 400 response before controller logic runs.

Expected error format: ValidationProblemDetails: a consistent response that identifies each invalid field and explains the reason, allowing client applications to show precise corrective guidance.

Day 5: API Hardening and Security Review

The final day added API-wide protections. Rate limiting restricts the number of requests within a time window and should be stricter on sensitive endpoints such as login to reduce brute-force and simple denial-of-service attacks. CORS was configured through a named policy that permits only known frontend origins; allowing every origin may be convenient during development but is unsafe for production.

HTTPS redirection, HSTS, and a restrictive Content-Security-Policy were identified as baseline production safeguards. Together they enforce encrypted transport, tell browsers to keep using HTTPS, and reduce exposure to browser-based injection and content risks.

EF Core LINQ queries parameterize values automatically, which prevents SQL injection when normal query APIs are used. The main danger is bypassing this protection with raw SQL assembled through string interpolation. Raw SQL should use FromSqlInterpolated or explicit parameters instead of interpolated FromSqlRaw strings.

Required Deliverables

- Working registration and login flow backed by ASP.NET Core Identity.
- JWT bearer authentication with token expiry and secure configuration.

- Protected CRUD routes with at least User and Admin roles, plus one policy beyond a simple role check.
- FluentValidation coverage for all Create and Update endpoints with structured validation errors.
- Rate limiting, named CORS policy, HTTPS/HSTS, and security headers configured.
- A Week 4 summary prepared for the mentor check-in.

Testing Checklist

1. Register a valid user and verify that weak or duplicate credentials produce meaningful errors.
2. Log in with valid credentials, decode the JWT to verify claims, and confirm expiry is enforced.
3. Call protected routes in Postman with no token, a valid User token, and an Admin token; confirm 401 and 403 behavior is correct.
4. Test every validation rule individually and verify the returned field-level 400 response.
5. Confirm a disallowed CORS origin fails, login limits trigger as configured, and no unsafe interpolated raw SQL remains.